A NEWBIE-FRIENDLY TOUR

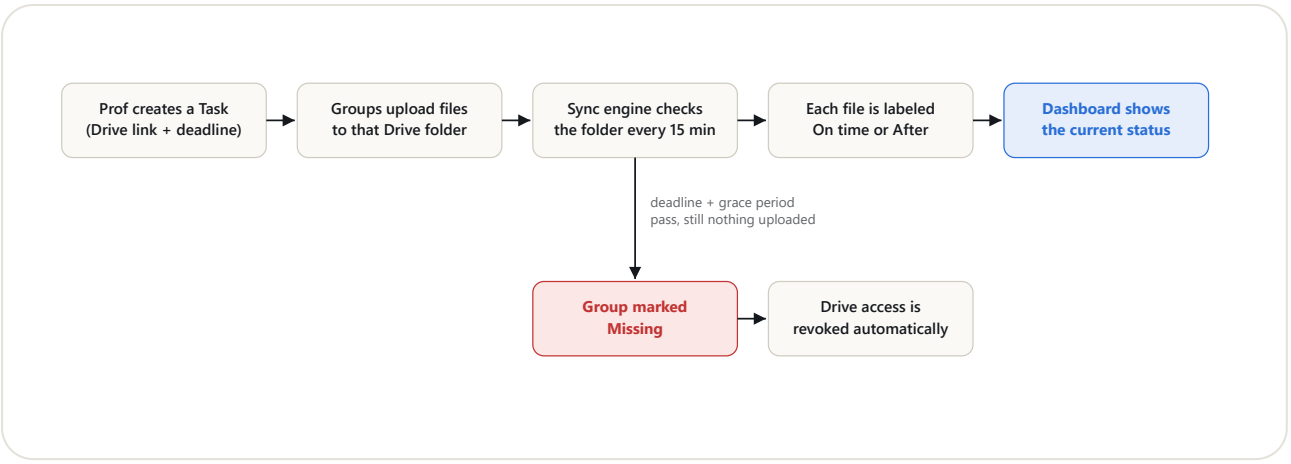
What actually happens when a group uploads a file?

This document explains how the Academic Submission Dashboard works, end to end, assuming no prior knowledge of the system. It covers the pieces that make it up, what changes between running it on your own laptop and hosting it online for free, and precisely what happens to one uploaded file between the moment it lands in Google Drive and the moment it shows up as On time or After on the dashboard.

The big picture

One end-to-end pass, before zooming into any single piece

The whole system exists to answer one question for a Prof, automatically and reliably: *who has actually submitted, and on time?* Rather than the Prof manually opening a Drive folder and checking file dates, the system watches that folder on the Prof's behalf and turns whatever it finds there into a dashboard. Here is the complete loop, from a task being created to a submission's final outcome:



— The full loop, and its two possible endings: a real submission gets timestamped and labeled, or silence past the deadline gets marked Missing and locked down.

Nothing in this loop is instant. The sync engine — the box doing the actual work — only runs on a timer, every 15 minutes by default, not the moment a file lands in Drive. Every box downstream of it, including the eventual Missing/lockout branch, only updates the next time that timer fires, or the moment the Prof clicks **Sync now** to force it early.

The building blocks

Six pieces, each with exactly one job

Set aside deadlines and audit trails for a moment, and the system is really just a handful of pieces passing information to each other. None of them do more than one job:

What you actually look at

The dashboard page (frontend)

The webpage itself — dropdowns for batch/semester/subject/task, the status tiles, the "Sync now" button. It's plain HTML, CSS, and JavaScript running inside your browser; it never stores or decides anything on its own, it only asks the server for data and displays whatever comes back.

The one that decides things

The server (backend)

A small Node.js program — code written in JavaScript that runs on a computer instead of inside a browser — that the dashboard page talks to. It answers questions like "what tasks exist?" and carries out actions like "create this task" or "extend this deadline."

The one that goes and checks

The sync engine

Part of the server, but worth naming separately: on a timer, it visits each task's Drive folder, notices new or renamed files, and works out whether each one is On time or After. This is the only piece that ever talks to Google Drive directly.

The permanent record

The database

Every batch, task, group, and submission the server has ever seen is stored here — written so that a record, once made, is never edited or deleted, only added to. That's what makes the audit report trustworthy: history can't quietly change after the fact.

The Prof's own Google sign-in

Google Drive, via "Connect Google Drive"

The dashboard never keeps its own copy of any file — it signs in as the Prof, with the Prof's explicit permission, and reads whatever Drive folders the Prof can already see. Students never interact with this system directly; they just upload to Drive as they always would.

Proof, kept alongside the record

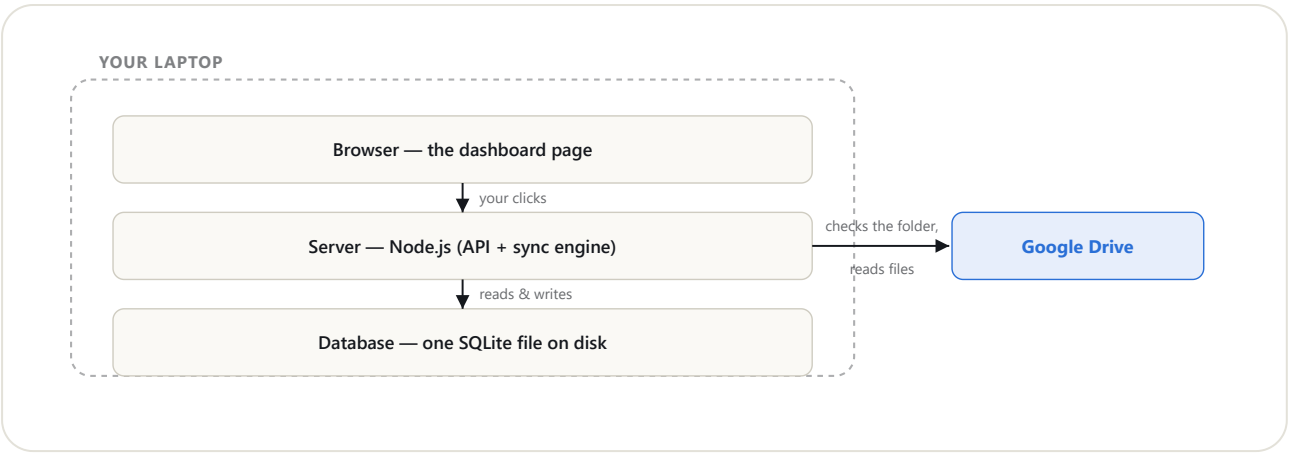
Evidence notes

A small, separate log where the Prof can attach a note — or a screenshot of the assignment email — to a task. It's stored the same durable way as everything else, so "I definitely sent this" has a receipt sitting right next to the record it backs up.

Running it on your own laptop

Local mode — the simplest way to run it

The easiest way to run this system is also the cheapest: on your own computer, with nothing to sign up for beyond a Google account. In this mode, three of the pieces above all live inside one folder on your machine, and only Google Drive is somewhere else — reached over the internet, the same way your browser reaches any website.



— All three inner pieces run on one machine; Google Drive is the only thing outside it. Simple and free, but only reachable while this laptop is on and the terminal window stays open.

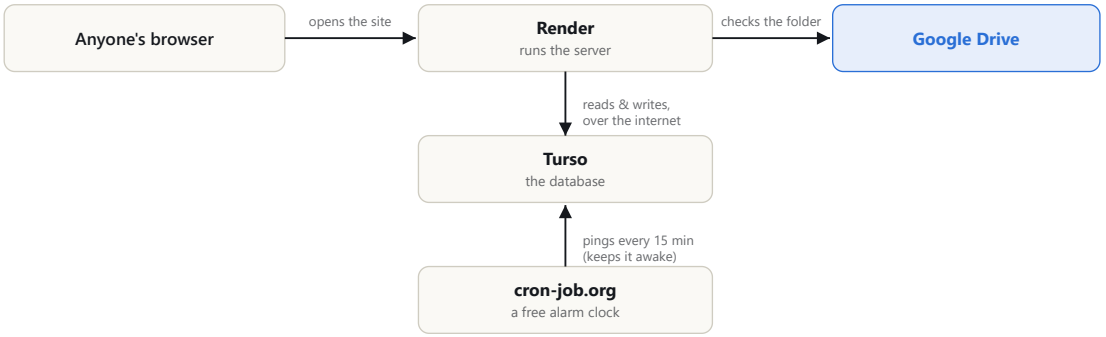
Getting to this point takes five steps, none of which involve creating a new account beyond Google itself:

1. **Install Node.js** — the program that runs the server, downloaded once from nodejs.org.
2. **Get the code** — clone or download the project folder onto this machine.
3. **Sign in with Google once** — via the "Connect Google Drive" button, so the server is allowed to read the Prof's Drive.
4. **Run one command** — `npm start` — which starts the server and creates the database file automatically the first time.
5. **Open the page** — `http://localhost:3000` in a browser. That address only means anything on this same computer.

Putting it online for free

Hosted mode — reachable from anywhere, still $0

Local mode has one real limitation baked into how it works: the dashboard only exists while this specific laptop is switched on and the server is running. If a co-instructor, or the Prof from their phone, needs to open the dashboard independently, it has to live somewhere that's always on — which local mode, by definition, is not. Hosted mode solves this by moving the same pieces onto free hosting services instead of onto your machine, three of them, each doing exactly one job:



— Same three inner pieces as local mode — Browser, Server, Database — but each now lives on its own free service and talks to the others over the internet. A fourth free service, cron-job.org, exists purely because free hosting falls asleep when nobody's visiting; its only job is to wake it back up on a schedule.

WHY THREE SERVICES, NOT ONE?

- **Render** runs the server code, but restarts it periodically and doesn't guarantee a private disk survives between restarts.
- **Turso** exists specifically so the database survives those restarts — it's a database that lives on the internet instead of on Render's disk.
- **cron-job.org** exists because Render's free tier falls asleep after 15 minutes with no visitors — without a periodic nudge, the automatic Drive-checking would quietly stop.

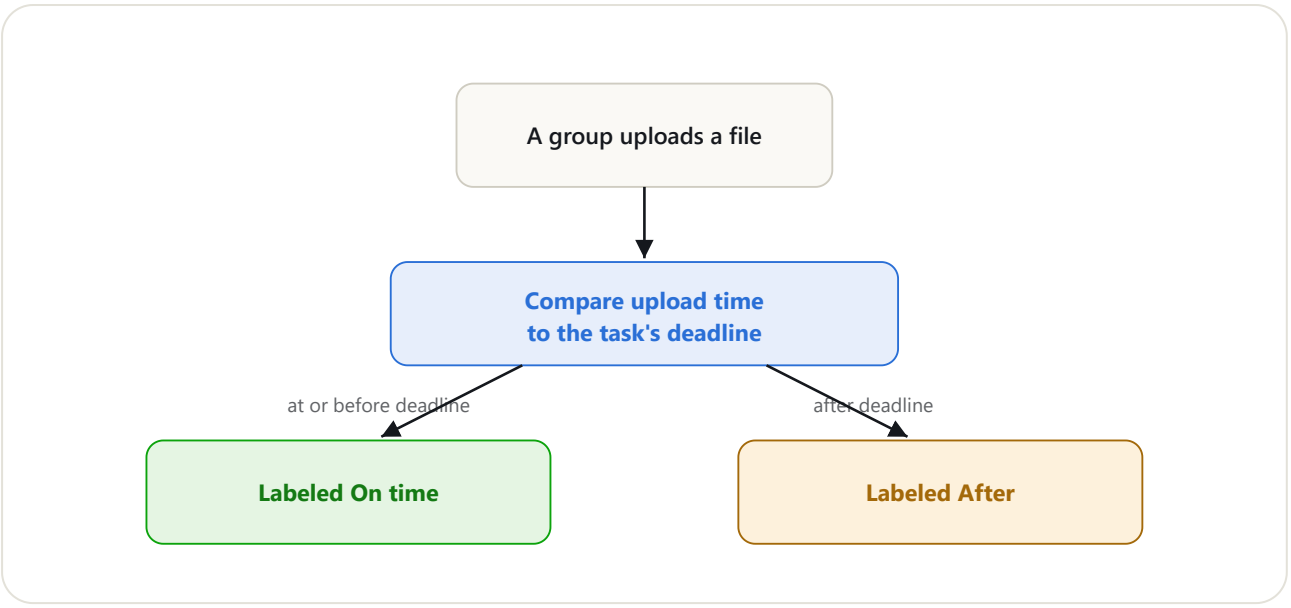
WHAT DOESN'T CHANGE

- The dashboard looks and behaves identically either way — hosting mode is invisible from inside the browser.
- The Prof still connects their own Google account the same way, through the same button.
- The database is still append-only — Turso only changes where records physically live, never how they're written.
- All three services have real, usable free tiers built for exactly this scale of tool.

A submission's journey

Following one file from upload to label

Zooming back in from infrastructure to a single file: here is exactly what decides whether an uploaded submission ends up **On time** or **After**.



— Two fixed facts feed this comparison: exactly when the file was uploaded, and the deadline set on the task. Change either input and the same comparison runs again.

That second input — the deadline — is not permanently fixed. If the Prof extends it later, the system re-runs this exact comparison for *every* submission already on that task, not just new ones. A file that was labeled After against the original 8:30 pm deadline can become On time the moment the deadline moves to 8:40 pm, because 8:40 is now the number its upload time is being compared against.

What never changes, even then, are the two underlying facts: the exact moment the file was uploaded, and the first moment the system noticed it existed. Those are permanent history. Only the *label* — the conclusion drawn by comparing that history to the current deadline — is free to update.

And if a group uploads *nothing* at all? Once the deadline plus a grace period (24 hours, by default) has fully passed, the system finalizes a **Missing** record for that group — permanently, and only once — and can automatically remove their upload access to the Drive folder.

Plain-English glossary

Every term on the dashboard, defined once

Batch `e.g. "2025–27"`

A cohort of students who started together — the top level of the hierarchy. Creating one automatically creates its 4 semesters.

Semester, Subject, Task

A strict nesting: a Batch has 4 Semesters, each Semester has Subjects, each Subject has Tasks. A **Task** is the concrete thing the Prof creates — one Drive link, one deadline.

Group `e.g. "group_3"`

Whoever uploaded a file, identified purely by the file's name — there's no separate roster to fill in beforehand. The first time "group_3.pdf" shows up, "group_3" is created automatically.

Sync

The act of the server visiting Drive, checking for new or renamed files, and updating the database. Happens automatically every 15 minutes, or immediately via the "Sync now" button.

On time / After / Missing

The three possible outcomes for a group on a task — see "A submission's journey" above for exactly how each one gets decided, and how the first two can change if a deadline is extended.

"Connect Google Drive" (OAuth)

The one-time sign-in that lets the server read — and, after a deadline, briefly manage access to — the Prof's Drive on their behalf, without the server ever needing the Prof's password.

Evidence note

A free-text note (optionally with a screenshot attached) the Prof can attach to a task — commonly proof that the assignment email was actually sent.

Audit report

A downloadable record of every submission that ever crossed a deadline (After or Missing), across every batch — pulled straight from the permanent log, unaffected by anything since.

Common questions

The things a newbie usually asks next

What's the actual difference between After and Missing?

After means a file did arrive, just later than the deadline — there's a real file to open. Missing means nothing ever arrived by the time the deadline plus grace period closed. They're tracked differently on purpose: Missing is a permanent, one-time judgment about silence, while After can still be corrected later by a deadline extension.

Where does my data actually live?

In local mode, it's one file on your own computer. In hosted mode, it's a Turso database somewhere on the internet — separate from, and not automatically synced with, whatever's on your laptop. They're two different datasets unless you deliberately point both at the same Turso database.

Is this really free, forever?

For the scale one Prof or a small department needs — yes. All three hosting services (Render, Turso, cron-job.org) have free tiers built for exactly this kind of small, low-traffic tool, and nothing in this setup requires a credit card to try.

What happens if my laptop loses internet mid-sync?

The current sync attempt fails safely and gets logged as an error — nothing gets half-written. The next scheduled sync, or a manual "Sync now" once you're back online, simply picks up where it left off.

Can a student see any of this?

No. Students never interact with the dashboard at all — they just upload to Drive exactly as they always would. The dashboard is a read-only window the Prof looks through afterward.

What if Drive access was already revoked before the deadline got extended?

The extension always re-labels existing submissions correctly. Re-granting Drive access that was already revoked, though, is a manual step for now — Google doesn't allow recreating a deleted permission automatically.

Academic Submission Dashboard — a batch/semester/subject/task tracker for Drive-based group assignments, with an append-only audit log.